Agent-Core — Build Plan

Status: DRAFT proposal (2026-07-04, afternoon). The **stack** below is locked; the **milestone order is a proposal** for Kaushik to reshape (evening session). Nothing else is committed.

Approach — build to learn

The survey's borrow/fix choices are **hypotheses**, not conclusions — an agent's behavior is emergent and can't be validated on paper. So we **build to learn**: one component at a time, each a **milestone**, each a small **design** → **build** → **observe** loop. We keep what works, fix what doesn't, and adjust this plan as we go.

Ordering rule: by *dependency*, not list-number — a minimal working agent first, then layer. **Observability early**, because it's how we judge "what worked."

Tech stack (locked 2026-07-04)

- **Language:** Python.
- **Agent loop:** our **own minimal loop** (*perceive* → *think* → *act* → *observe*) — not a framework. Study-and-rebuild; we own every decision point (controllability-by-construction).
- **Provider transport (hybrid):**
  - **LiteLLM** → local vLLM model + any true-API provider.
  - **CLI-wrappers** → **subscription** Claude (Claude Code / Agent SDK on Claude Max) and OpenAI (Codex on ChatGPT sub). The Velhari pattern, rebuilt — flat sub cost, no per-token API.
- **Local serving:** vLLM on the DGX Spark (e.g. Llama-class 70B).
- **Later-milestone tech (not locked):** memory = CM-kind (rebuild from CM); persona = `persona.md`; skills = agentskills.io / `SKILL.md` progressive disclosure.

Components

The 9 from the requirement, **plus the Router** (emerged 2026-07-04): the intent → provider allocation brain (the loop states *intent*; the Router picks the provider by policy — privacy gate → local; bulk → local; hard → subscription; high fan-out → local; + consortium override + live fallback).

Core: 1 persona/thinking · 2 memory · 3 orchestrator · 4 connectors · 5 tools · 6 skills · 7 interface · 8 self-correction · 9 observability · **R Router**. Later-phase: multi-user/permissions · action guardrails.

Milestones (PROPOSED — dependency-ordered; reshape freely)

- **M1 — Walking skeleton.** Persona + minimal agent loop + CLI interface + a *minimal* Router (subscription-Claude first). Something you can talk to that thinks and takes one basic action. *(K liked this as the start.)*
- **M2 — Observability.** Watch it think — reasoning trace surfaced. Early on purpose, so every later milestone is judgeable (and it's the trust core).
- **M3 — Memory.** CM-kind persistence; learns across sessions.

- **M4 — Tools.** Structured tools + a code escape-hatch (hybrid action space). The escape-hatch runs **working-dir-scoped + an approval gate** for destructive ops (Claude-Code-style permissions) — *not* full sandboxing yet. Full isolation (container/VM) moves to **C11**, triggered when untrusted input flows in (connectors) or on multi-user deployment.
- **M5 — Skills.** agentskills.io progressive disclosure; self-authoring.
- **M6 — Router (full).** The complete intent→provider policy (local / sub / API, privacy gate, volume, fallback). Minimal version was seeded in M1.
- **M7 — Orchestrator.** Multi-provider consortium — the "committee." Velhari-pattern.
- **M8 — Self-correction.** Capture lessons + deliver them at the decision point.
- **M9 — Connectors.** Email / Slack / Drive knowledge intake.
- **Later:** multi-user / permissions; action guardrails.

Decisions (resolved 2026-07-04 evening)

- **Milestone order** — LOCKED as proposed (M1 → M9, dependency-ordered).
- **Router** — LOCKED as its **own component**: a thin *seam* at M1 (single-provider stub), grown to a full component by M6/M7. Rationale: two consumers (the loop + the orchestrator) need the same allocation brain; a standalone Router keeps the loop provider-agnostic and gives the cost/privacy/capability policy one home.
- **M1 interface** — LOCKED **CLI only** (no web surface at M1).
- **Persona + memory** — persona file = `persona.md`; memory = **rebuild CM** (not adapt).
- **M4 sandboxing** — working-dir scope + approval-gate at M4; full isolation → **C11** (triggered by untrusted input or multi-user).

Each milestone runs its own design → build → observe pass; the plan flexes as we learn.